N+1 problem

> **N+1 happens when the ORM executes 1 query to load parent entities then N additional queries to load related entities lazily.**
>
> N+1 is a data-access design problem, not a Hibernate problem.

So total queries = **1 + N**

This is **not a bug**, it's a **default ORM behavior**.

"N+1 is not specific to Hibernate. It's a data access anti-pattern where you first load a list of parent entities, then execute one query per parent to load related data. ORMs make it easier to fall into because queries are triggered implicitly."

Simple domain (classic interview example)

Entities

```java
@Entity
public class User {
    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "user", fetch = FetchType.LAZY) // LAZY by default
    private List<Order> orders;
}
```

```java
@Entity
public class Order {
    @Id
    private Long id;

    private BigDecimal amount;

    @ManyToOne
    private User user;
}
```

The Problem — Step by Step

The Broken Code

```java
// Repository
List<User> users = userRepository.findAll();
```

```
// Iterating — triggers N additional queries
for (User user : users) {
    System.out.println(user.getName() + " : " +
user.getOrders().size());
    //                                              ^^^^^^^^^^^^^^^^
    //                           getOrders() triggers a SELECT per
user
}

```

SQL Generated (10 users → 11 queries)

```
-- Query 1: load all users
SELECT * FROM user;

-- Query 2 to 11: one per user (N queries)
SELECT * FROM order WHERE user_id = 1;
SELECT * FROM order WHERE user_id = 2;
SELECT * FROM order WHERE user_id = 3;

```

Impact

With 100 users → 101 queries. With 1000 users → 1001 queries. Each query has network latency + DB overhead. In production this causes severe performance degradation under load.

Solutions — From Simple to Best Practice

Solution 1 — JOIN FETCH (JPQL)

Forces Hibernate to load the association in a single JOIN query.

```
// Repository
@Query("SELECT u FROM User u JOIN FETCH u.orders")
List<User> findAllWithOrders();

// SQL generated — 1 single query
-- SELECT u.*, o.* FROM user u
-- INNER JOIN order o ON o.user_id = u.id

```

Limitation

JOIN FETCH with @OneToMany returns duplicate parent rows — wrap result in a Set<> or use DISTINCT: "SELECT DISTINCT u FROM User u JOIN FETCH u.orders"

Solution 2 — @EntityGraph

Declares which associations to eagerly load without writing JPQL. Spring Data generates the JOIN automatically.

```
// On the Repository method
@EntityGraph(attributePaths = {"orders"})
```

```
List<User> findAll();

// Or define a named graph on the Entity
@NamedEntityGraph(
    name = "User.withOrders",
    attributeNodes = @NamedAttributeNode("orders")
)
@Entity
public class User { ... }

// Then reference it
@EntityGraph("User.withOrders")
List<User> findAll();

```

@EntityGraph tells Spring Data to eagerly fetch the specified associations for this specific query — it generates a LEFT JOIN FETCH automatically. Result is identical to JOIN FETCH but you avoid writing JPQL. Advantage: you can apply it per-method without modifying the entity or writing custom queries. Both produce 1 query.

Solution 3 — DTO Projection (Best Performance)

Instead of loading entities at all, project only the fields you need directly into a DTO. No entity lifecycle, no dirty checking, minimal memory footprint.

```
// DTO
public record UserOrderDto(Long userId, String name, Long orderId,
BigDecimal amount) {}

// Repository
@Query("""
    SELECT new ma.app.dto.UserOrderDto(u.id, u.name, o.id, o.amount)
    FROM User u JOIN u.orders o
    """)
List<UserOrderDto> findAllAsDto();

// SQL generated — 1 query, only selected columns
-- SELECT u.id, u.name, o.id, o.amount
-- FROM user u INNER JOIN order o ON o.user_id = u.id

```

Solution 4 — @BatchSize (Hibernate-specific)

Instead of 1 query per parent, Hibernate groups the IN clauses: 1 + ceil(N/batchSize) queries.

```
@Entity
public class User {

    @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
    @BatchSize(size = 20)  // loads 20 users' orders per query
    private List<Order> orders;
}

-- Instead of 100 queries for 100 users:
-- SELECT * FROM order WHERE user_id IN (1,2,3,...,20);  -- 5 queries
total

```

5. Solution Comparison

Solution	Queries	Flexibility	Performance	Complexity
JOIN FETCH	1	High	Good	Low
@EntityGraph	1	High	Good	Low
DTO Projection	1 (partial)	Medium	**Best**	Medium
@BatchSize	1 + N/batch	High	Moderate	Low
No fix (N+1)	1 + N	High	**Poor**	None

6. How to Detect N+1

Enable SQL Logging

```yaml
# application.yml
spring:
  jpa:
    show-sql: true
    properties:
      hibernate:
        format_sql: true

logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE  # shows bind parameters

```

Use Hibernate Statistics

```yaml
spring:
  jpa:
    properties:
      hibernate:
        generate_statistics: true

# Logs total queries per session — look for high query counts

```

Follow-up Questions & Answers

Q: When does N+1 actually occur?

- When iterating a lazily-loaded collection outside its owning transaction
- When LAZY-loaded data is accessed after the session is closed (also causes LazyInitializationException)

Q: Is EAGER fetch the solution?

No — EAGER is worse

EAGER always loads the collection even when you don't need it. It trades N+1 for unnecessary data loading on every query. The correct solution is LAZY + explicit JOIN FETCH where needed.

Q: What is LazyInitializationException?

- Occurs when a LAZY collection is accessed after the Hibernate session is closed
- Common in REST APIs: entity loaded in @Transactional service, then serialized in controller (outside transaction)
- Fix: use DTOs and map before the transaction closes, or use Open Session in View (not recommended in production)

```
@Transactional method        Controller
─────────────────────        ──────────
findById(id)    ✓ session open
return user     ✓ session open
                ↓
            session CLOSED  ← transaction boundary
                ↓
user.getOrders()            ✗ no session → exception
```

The entity left the transaction boundary as a **detached** object. The `orders` proxy has no session to execute the SQL against.

Q: Can @BatchSize fully replace JOIN FETCH?

No. @BatchSize reduces query count but doesn't eliminate it. For reporting or bulk loading, JOIN FETCH or DTO projections are always faster.